

```

import json
import logging
import os
import secrets
import string
import urllib.error
import urllib.parse
import urllib.request
from contextlib import contextmanager
from datetime import datetime
from typing import Any, Dict, List, Optional

from flask import Flask, jsonify, request
import pycopg
from pycopg.rows import dict_row

# =====
# CONFIG
# =====

BOT_TOKEN = os.getenv("BOT_TOKEN", "").strip()
DATABASE_URL = os.getenv("DATABASE_URL", "").strip()
WEBHOOK_URL = os.getenv("WEBHOOK_URL", "").strip().rstrip("/")
WEBHOOK_SECRET = os.getenv("WEBHOOK_SECRET", "").strip()
ADMIN_KEY = os.getenv("ADMIN_KEY", "").strip()
PORT = int(os.getenv("PORT", "10000"))

TELEGRAM_API = f"https://api.telegram.org/bot{BOT_TOKEN}" if BOT_TOKEN else ""

app = Flask(__name__)
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("strategy-game")

# =====
# TRANSLATIONS
# =====

TEXTS = {
    "fa": {
        "welcome": "از دکمه‌های پایین برای ساخت یا پیوستن به.\n\nبه بازی استراتژی چندنفره خوش آمدی {name}! سلام",
        "welcome_private": ".بازی را می‌توانی از اینجا مدیریت کنی و بعد در گروه وارد لابی شوی {name}! سلام",
        "choose_language": "زبان را انتخاب کن",
        "language_saved": "زبان ذخیره شد",
        "help": (
            "راهنما\n\n"
            ".\nساخت بازی: یک لابی جدید می‌سازد 🎮"
            ".\nپیوستن به بازی: با کد ۶ حرفی وارد لابی می‌شوی 🗝️"
        )
    }
}

```

"\nکشور: کشور آزاد را برای بازی انتخاب می‌کنی 🌐"
"\nبازی من: وضعیت بازی فعلی را می‌بینی 📋"
"\n\nقوانین: اقتصاد، تحقیق، صنعت و دیپلماسی را مدیریت کن و امتیاز بیشتری بگیر 📖"
".حداقل ۲ بازیکن برای شروع لازم است"

),

"rules": (

"\n\nقوانین نسخه فعلی"
"\nهر بازیکن یک کشور انتخاب می‌کند."
"\nبازی نوبتی است."
"\nدر هر نوبت ۳ اقدام داری."
"\nاقدام‌ها: ساخت صنعت، تحقیق، تجارت و دیپلماسی."
"\nپایان هر نوبت درآمد و منابع پایه تولید می‌شود."
"\nبازی در ۳۰ نوبت کل تمام می‌شود."
".امتیاز بر اساس پول، صنعت، تحقیق، ثبات و جمعیت محاسبه می‌شود."

),

"unknown": "دستور یا پیام نامشخص است. از دکمه‌های پایین استفاده کن",

"need_group": "این گزینه را بهتر است داخل گروهی که بازی در آن برگزار می‌شود اجرا کنی",

"game_created": "سازنده: {name}\n\nکد بازی: <code>{code}</code>\n\nبازی ساخته شد" 🟢

",
بقیه بازیکن‌ها بفرست

"game_already": "<code>{code}</code>: تو همین الان در یک بازی فعال هستی",

"join_enter_code": "کد ۶ حرفی بازی را بفرست",

"invalid_code": "کد باید دقیقاً ۶ حرف یا عدد باشد",

"game_not_found": "چنین بازی‌ای پیدا نشد یا بازی دیگر در وضعیت انتظار نیست",

"joined": "وارد بازی شدی" 🟢\n\n{game}",

"already_in_game": "تو از قبل داخل این بازی هستی",

"country_open": "یک کشور آزاد انتخاب کن",

"country_taken": "این کشور قبلاً توسط بازیکن دیگری انتخاب شده است",

"country_saved": "✅ برای تو ثبت شد {country} کشور",

"need_country": "قبل از شروع، همه بازیکن‌ها باید کشورشان را انتخاب کنند",

"not_owner": "فقط سازنده بازی می‌تواند آن را شروع کند",

"need_players": "برای شروع حداقل ۲ بازیکن لازم است",

"game_started": "{player}: نوبت اول 🚀\n\nبازی شروع شد",

"not_your_turn": "{player}: الان نوبت تو نیست. نوبت",

"not_running": "بازی در حال اجرا نیست",

"action_done": "✅\n\n{summary} اقدام انجام شد",

"not_enough_money": "{money}: پول کافی نیست. موجودی فعلی",

"not_enough_actions": "اقدام‌های این نوبت تمام شده‌اند. نوبت را تمام کن",

"turn_ended": "{player}: تمام شد. نوبت بعد {turn} نوبت",

"game_finished": "🏁\n\n{results} بازی تمام شد",

"no_game": "فعلاً در هیچ بازی فعالی نیستی",

"status": "{game}",

"players_title": "بازیکنان",

"lobby": "لابی بازی",

"running": "بازی در حال اجرا",

"finished": "بازی تمام شده",

"waiting": "در انتظار بازیکن",

"owner": "سازنده",

"turn": "نوبت",

"round": "دوره",

"actions": "اقدام‌های باقی‌مانده",

```

"money": "پول",
"industry": "صنعت",
"science": "تحقیق",
"stability": "ثبات",
"population": "جمعیت",
"country": "کشور",
"no_country": "انتخاب نشده",
"start_game": "شروع بازی",
"choose_country": "🌐 انتخاب کشور",
"join_button": "🔑 پیوستن به بازی",
"create_button": "🎮 ساخت بازی",
"my_game": "📋 بازی من",
"language_button": "🌐 زبان",
"help_button": "❓ راهنما",
"rules_button": "📖 قوانین",
"game_menu": "🎯 بازی",
"build": "🏠 ساخت صنعت",
"research": "🔬 تحقیق",
"trade": "💰 تجارت",
"diplomacy": "📧 دیپلماسی",
"end_turn": "🏁 پایان نوبت",
"back": "⬅️ بازگشت",
"cancel": "❌ لغو",
"private_tip": ". برای هماهنگی بهتر، این پیام را داخل گروه بازی اجرا کن",
"admin_only": ". این مسیر مدیریتی است",
"migrated": "Database migration completed.",
},
"en": {
  "welcome": "Hi {name}!\n\nWelcome to the multiplayer strategy game.\n\nUse the buttons below to create or join a game.",
  "welcome_private": "Hi {name}!\n\nYou can manage the game here and then play in your group.",
  "choose_language": "Choose your language:",
  "language_saved": "Language saved.",
  "help": (
    "Help\n\n"
    "🎮 Create Game: create a new lobby.\n"
    "🔑 Join Game: join a lobby with a 6-character code.\n"
    "🌐 Country: choose an available country.\n"
    "📋 My Game: show your current game.\n"
    "📖 Rules: manage economy, research, industry and diplomacy.\n\n"
    "At least 2 players are required to start."
  ),
  "rules": (
    "Current rules\n\n"
    "• Each player chooses one country.\n"
    "• The game is turn-based.\n"
    "• You have 3 actions per turn.\n"
    "• Actions: industry, research, trade and diplomacy.\n"
    "• Each completed turn generates base income and resources.\n"
  )
}

```

```

    "• The game ends after 30 total turns.\n"
    "• Score uses money, industry, research, stability and population."
),
"unknown": "I didn't understand that. Please use the buttons.",
"need_group": "This option works best inside the group where the game will be played.",
"game_created": "Game created ✅\n\nGame code: <code>{code}</code>\nOwner:
{name}\n\nSend the code to the other players.",
"game_already": "You are already in an active game: <code>{code}</code>.",
"join_enter_code": "Send the 6-character game code.",
"invalid_code": "The code must be exactly 6 letters or digits.",
"game_not_found": "That game was not found or is no longer waiting.",
"joined": "You joined the game ✅\n\n{game}",
"already_in_game": "You are already in this game.",
"country_open": "Choose an available country:",
"country_taken": "That country has already been chosen.",
"country_saved": "Country {country} saved ✅",
"need_country": "Everyone must choose a country before the game can start.",
"not_owner": "Only the game owner can start the game.",
"need_players": "At least 2 players are required.",
"game_started": "Game started 🚀\n\nFirst turn: {player}",
"not_your_turn": "It is not your turn. Current turn: {player}",
"not_running": "The game is not running.",
"action_done": "Action completed ✅\n\n{summary}",
"not_enough_money": "Not enough money. Current balance: {money}",
"not_enough_actions": "You have no actions left. End your turn.",
"turn_ended": "Turn {turn} ended. Next: {player}",
"game_finished": "Game finished 🏁\n\n{results}",
"no_game": "You are not in an active game.",
"status": "{game}",
"players_title": "Players",
"lobby": "Game lobby",
"running": "Game running",
"finished": "Game finished",
"waiting": "Waiting for players",
"owner": "Owner",
"turn": "Turn",
"round": "Round",
"actions": "Actions left",
"money": "Money",
"industry": "Industry",
"science": "Research",
"stability": "Stability",
"population": "Population",
"country": "Country",
"no_country": "Not selected",
"start_game": "Start Game",
"choose_country": "🌍 Choose Country",
"join_button": "🔑 Join Game",
"create_button": "🎮 Create Game",
"my_game": "📄 My Game",

```

```

"language_button": "🌐 Language",
"help_button": "❓ Help",
"rules_button": "📖 Rules",
"game_menu": "🎯 Game",
"build": "🏢 Build Industry",
"research": "🔬 Research",
"trade": "💰 Trade",
"diplomacy": "🤝 Diplomacy",
"end_turn": "▶▶ End Turn",
"back": "⬅️ Back",
"cancel": "❌ Cancel",
"private_tip": "For coordination, run this message inside your game group.",
"admin_only": "This is an administrative route.",
"migrated": "Database migration completed.",
},
}

COUNTRIES = {
    "IR": {"fa": "ایران", "en": "Iran", "industry": 11, "science": 4, "stability": 68,
"population": 85},
    "US": {"fa": "ایالات متحده", "en": "United States", "industry": 18, "science": 10, "stability":
72, "population": 90},
    "CN": {"fa": "چین", "en": "China", "industry": 17, "science": 8, "stability": 74,
"population": 95},
    "DE": {"fa": "آلمان", "en": "Germany", "industry": 16, "science": 9, "stability": 78,
"population": 55},
    "FR": {"fa": "فرانسه", "en": "France", "industry": 14, "science": 8, "stability": 76,
"population": 52},
    "TR": {"fa": "ترکیه", "en": "Türkiye", "industry": 12, "science": 5, "stability": 66,
"population": 60},
    "JP": {"fa": "ژاپن", "en": "Japan", "industry": 15, "science": 10, "stability": 82,
"population": 50},
    "IN": {"fa": "هند", "en": "India", "industry": 13, "science": 7, "stability": 67,
"population": 100},
    "BR": {"fa": "برزیل", "en": "Brazil", "industry": 10, "science": 4, "stability": 65,
"population": 72},
    "CA": {"fa": "کانادا", "en": "Canada", "industry": 11, "science": 8, "stability": 80,
"population": 38},
    "EG": {"fa": "مصر", "en": "Egypt", "industry": 9, "science": 3, "stability": 63,
"population": 68},
    "KR": {"fa": "کره جنوبی", "en": "South Korea", "industry": 15, "science": 10, "stability": 79,
"population": 48},
}

# =====
# HELPERS
# =====

def t(user_id:int, key: str, **kwargs: Any) -> str:

```

```

language = get_language(user_id)
value = TEXTS.get(language, TEXTS["en"]).get(key, key)
return value.format(**kwargs)

def country_name(code: Optional[str], language: str = "en") -> str:
    if not code or code not in COUNTRIES:
        return TEXTS.get(language, TEXTS["en"])["no_country"]
    return COUNTRIES[code][language]

def display_name(user: Dict[str, Any]) -> str:
    first = (user.get("first_name") or "").strip()
    last = (user.get("last_name") or "").strip()
    username = (user.get("username") or "").strip()
    full = " ".join(x for x in (first, last) if x).strip()
    if full:
        return full
    if username:
        return f"@{username}"
    return str(user.get("id", "Player"))

@contextmanager
def get_conn():
    if not DATABASE_URL:
        raise RuntimeError("DATABASE_URL is missing")
    conn = psycopg2.connect(DATABASE_URL, row_factory=dict_row)
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

def generate_code(length: int = 6) -> str:
    alphabet = string.ascii_uppercase + string.digits
    return "".join(secrets.choice(alphabet) for _ in range(length))

# =====
# DATABASE / MIGRATION
# =====

def column_exists(cur, table: str, column: str) -> bool:
    cur.execute(

```

```

"""
SELECT 1
FROM information_schema.columns
WHERE table_schema = 'public'
      AND table_name = %s
      AND column_name = %s
      """
(table, column),
)
return cur.fetchone() is not None

```

```

def add_column_if_missing(cur, table: str, column: str, definition: str):
    if not column_exists(cur, table, column):
        cur.execute(f'ALTER TABLE "{table}" ADD COLUMN "{column}" {definition}')
        logger.info("Added missing column %s.%s", table, column)

def init_db():
    with get_conn() as conn:
        with conn.cursor() as cur:
            # New/full schema. IF NOT EXISTS also keeps existing deployments alive.
            cur.execute(
                """
                CREATE TABLE IF NOT EXISTS user_preferences (
                    user_id BIGINT PRIMARY KEY,
                    language TEXT NOT NULL DEFAULT 'fa',
                    pending_action TEXT,
                    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
                )
                """
            )
            cur.execute(
                """
                CREATE TABLE IF NOT EXISTS games (
                    id SERIAL PRIMARY KEY,
                    code VARCHAR(6) UNIQUE NOT NULL,
                    owner_id BIGINT,
                    status TEXT NOT NULL DEFAULT 'waiting',
                    total_turn INTEGER NOT NULL DEFAULT 0,
                    current_player_id BIGINT,
                    created_at TIMESTAMP NOT NULL DEFAULT NOW(),
                    updated_at TIMESTAMP NOT NULL DEFAULT NOW()
                )
                """
            )
            cur.execute(
                """
                CREATE TABLE IF NOT EXISTS players (
                    id SERIAL PRIMARY KEY,

```

```

        game_id INTEGER NOT NULL REFERENCES games(id) ON DELETE CASCADE,
        user_id BIGINT NOT NULL,
        username TEXT,
        first_name TEXT,
        chat_id BIGINT,
        country VARCHAR(2),
        money INTEGER NOT NULL DEFAULT 1000,
        industry INTEGER NOT NULL DEFAULT 10,
        science INTEGER NOT NULL DEFAULT 0,
        stability INTEGER NOT NULL DEFAULT 70,
        population INTEGER NOT NULL DEFAULT 50,
        actions_left INTEGER NOT NULL DEFAULT 3,
        score INTEGER NOT NULL DEFAULT 0,
        joined_at TIMESTAMP NOT NULL DEFAULT NOW(),
        UNIQUE(game_id, user_id)
    )
    """"
)

# Automatic migration for the older schema that caused owner_id errors.
add_column_if_missing(cur, "user_preferences", "pending_action", "TEXT")
add_column_if_missing(cur, "user_preferences", "updated_at", "TIMESTAMP NOT NULL
DEFAULT NOW()")

add_column_if_missing(cur, "games", "owner_id", "BIGINT")
add_column_if_missing(cur, "games", "total_turn", "INTEGER NOT NULL DEFAULT 0")
add_column_if_missing(cur, "games", "current_player_id", "BIGINT")
add_column_if_missing(cur, "games", "created_at", "TIMESTAMP NOT NULL DEFAULT
NOW()")

add_column_if_missing(cur, "games", "updated_at", "TIMESTAMP NOT NULL DEFAULT
NOW()")

add_column_if_missing(cur, "players", "username", "TEXT")
add_column_if_missing(cur, "players", "first_name", "TEXT")
add_column_if_missing(cur, "players", "chat_id", "BIGINT")
add_column_if_missing(cur, "players", "country", "VARCHAR(2)")
add_column_if_missing(cur, "players", "money", "INTEGER NOT NULL DEFAULT 1000")
add_column_if_missing(cur, "players", "industry", "INTEGER NOT NULL DEFAULT 10")
add_column_if_missing(cur, "players", "science", "INTEGER NOT NULL DEFAULT 0")
add_column_if_missing(cur, "players", "stability", "INTEGER NOT NULL DEFAULT 70")
add_column_if_missing(cur, "players", "population", "INTEGER NOT NULL DEFAULT 50")
add_column_if_missing(cur, "players", "actions_left", "INTEGER NOT NULL DEFAULT 3")
add_column_if_missing(cur, "players", "score", "INTEGER NOT NULL DEFAULT 0")
add_column_if_missing(cur, "players", "joined_at", "TIMESTAMP NOT NULL DEFAULT
NOW()")

# Repair owner_id for legacy games where a player row already exists.
cur.execute(
    """"
    UPDATE games g

```

```

        SET owner_id = x.user_id
    FROM (
        SELECT DISTINCT ON (game_id) game_id, user_id
        FROM players
        ORDER BY game_id, id
    ) x
    WHERE g.id = x.game_id
        AND g.owner_id IS NULL
    """
)

cur.execute("CREATE INDEX IF NOT EXISTS idx_games_status ON games(status)")
cur.execute("CREATE INDEX IF NOT EXISTS idx_players_game ON players(game_id)")
cur.execute("CREATE INDEX IF NOT EXISTS idx_players_user ON players(user_id)")
cur.execute("CREATE INDEX IF NOT EXISTS idx_games_owner ON games(owner_id)")

```

```

logger.info("Database is ready")

```

```

# =====
# USER SETTINGS
# =====

```

```

def get_language(user_id: int) -> str:
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT language FROM user_preferences WHERE user_id = %s",
(user_id,))

                row = cur.fetchone()
                return row["language"] if row and row.get("language") in TEXTS else "fa"
    except Exception:
        logger.exception("Failed to read language")
        return "fa"

```

```

def set_language(user_id: int, language: str):
    if language not in TEXTS:
        return
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO user_preferences (user_id, language, updated_at)
                VALUES (%s, %s, NOW())
                ON CONFLICT (user_id)
                DO UPDATE SET language = EXCLUDED.language,
                    updated_at = NOW()
                """,

```

```

        (user_id, language),
    )

def get_pending_action(user_id: int) -> Optional[str]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT pending_action FROM user_preferences WHERE user_id = %s",
                (user_id,))
            row = cur.fetchone()
            return row["pending_action"] if row else None

def set_pending_action(user_id: int, action: Optional[str]):
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO user_preferences (user_id, language, pending_action, updated_at)
                VALUES (%s, COALESCE((SELECT language FROM user_preferences WHERE user_id = %s),
                'fa'), %s, NOW())
                ON CONFLICT (user_id)
                DO UPDATE SET pending_action = EXCLUDED.pending_action,
                               updated_at = NOW()
                """,
                (user_id, user_id, action),
            )

# =====
# GAME DATABASE
# =====

def get_game_by_code(code: str) -> Optional[Dict[str, Any]]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT * FROM games WHERE code = %s", (code.upper(),))
            return cur.fetchone()

def get_user_game(user_id: int) -> Optional[Dict[str, Any]]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                """
                SELECT g.*
                FROM games g
                JOIN players p ON p.game_id = g.id
                WHERE p.user_id = %s
            """
            )

```

```

        AND g.status IN ('waiting', 'running')
    ORDER BY g.id DESC
    LIMIT 1
    """
    (user_id,),
)
return cur.fetchone()

```

```

def get_players(game_id: int) -> List[Dict[str, Any]]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM players WHERE game_id = %s ORDER BY id",
                (game_id,),
            )
            return list(cur.fetchall())

```

```

def get_player(game_id: int, user_id: int) -> Optional[Dict[str, Any]]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute(
                "SELECT * FROM players WHERE game_id = %s AND user_id = %s",
                (game_id, user_id),
            )
            return cur.fetchone()

```

```

def create_game(user: Dict[str, Any], chat_id: int) -> Dict[str, Any]:
    existing = get_user_game(int(user["id"]))
    if existing:
        raise ValueError("already_in_game")

    with get_conn() as conn:
        with conn.cursor() as cur:
            for _ in range(20):
                code = generate_code()
                try:
                    cur.execute(
                        """
                        INSERT INTO games (code, owner_id, status, total_turn)
                        VALUES (%s, %s, 'waiting', 0)
                        RETURNING *
                        """
                        (code, int(user["id"])),
                    )
                    game = cur.fetchone()
                    break
            except psycopg.errors.UniqueViolation:

```

```

        conn.rollback()
        continue
    else:
        raise RuntimeError("Could not generate a unique game code")

    cur.execute(
        """
        INSERT INTO players (
            game_id, user_id, username, first_name, chat_id,
            money, industry, science, stability, population,
            actions_left, score
        )
        VALUES (%s, %s, %s, %s, %s, 1000, 10, 0, 70, 50, 3, 0)
        """,
        (
            game["id"],
            int(user["id"]),
            user.get("username"),
            user.get("first_name"),
            chat_id,
        ),
    )
    return game

```

```

def join_game(game_id: int, user: Dict[str, Any], chat_id: int):
    existing = get_user_game(int(user["id"]))
    if existing:
        if existing["id"] == game_id:
            raise ValueError("already_in_game")
        raise ValueError("already_in_game")

    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT * FROM games WHERE id = %s FOR UPDATE", (game_id,))
            game = cur.fetchone()
            if not game or game["status"] != "waiting":
                raise ValueError("game_not_found")

            cur.execute(
                "SELECT 1 FROM players WHERE game_id = %s AND user_id = %s",
                (game_id, int(user["id"])),
            )
            if cur.fetchone():
                raise ValueError("already_in_game")

            cur.execute("SELECT COUNT(*) AS n FROM players WHERE game_id = %s", (game_id,))
            count = cur.fetchone()["n"]
            if count >= 12:
                raise ValueError("game_full")

```

```

cur.execute(
    """
    INSERT INTO players (
        game_id, user_id, username, first_name, chat_id,
        money, industry, science, stability, population,
        actions_left, score
    )
    VALUES (%s, %s, %s, %s, %s, 1000, 10, 0, 70, 50, 3, 0)
    """,
    (
        game_id,
        int(user["id"]),
        user.get("username"),
        user.get("first_name"),
        chat_id,
    ),
)

```

```

def choose_country(game_id: int, user_id: int, code: str) -> bool:

```

```

    code = code.upper()

```

```

    if code not in COUNTRIES:

```

```

        return False

```

```

    with get_conn() as conn:

```

```

        with conn.cursor() as cur:

```

```

            cur.execute("SELECT status FROM games WHERE id = %s", (game_id,))

```

```

            game = cur.fetchone()

```

```

            if not game or game["status"] != "waiting":

```

```

                return False

```

```

            cur.execute(

```

```

                "SELECT 1 FROM players WHERE game_id = %s AND country = %s AND user_id <> %s",
                (game_id, code, user_id),
            )

```

```

            if cur.fetchone():

```

```

                raise ValueError("country_taken")

```

```

            stats = COUNTRIES[code]

```

```

            cur.execute(

```

```

                """

```

```

                UPDATE players

```

```

                SET country = %s,

```

```

                    industry = %s,

```

```

                    science = %s,

```

```

                    stability = %s,

```

```

                    population = %s

```

```

                WHERE game_id= %s AND user_id = %s

```

```

                """,

```

```

        (
            code,
            stats["industry"],
            stats["science"],
            stats["stability"],
            stats["population"],
            game_id,
            user_id,
        ),
    )
    return cur.rowcount > 0

```

```

def start_game(game_id: int, user_id: int) -> Dict[str, Any]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT * FROM games WHERE id = %s FOR UPDATE", (game_id,))
            game = cur.fetchone()
            if not game:
                raise ValueError("game_not_found")
            if game["owner_id"] != user_id:
                raise ValueError("not_owner")
            if game["status"] != "waiting":
                raise ValueError("not_running")

            cur.execute("SELECT * FROM players WHERE game_id = %s ORDER BY id", (game_id,))
            players = list(cur.fetchall())
            if len(players) < 2:
                raise ValueError("need_players")
            if any(not p.get("country") for p in players):
                raise ValueError("need_country")

            first_player = players[0]
            cur.execute(
                """
                UPDATE games
                SET status = 'running',
                    total_turn = 1,
                    current_player_id = %s,
                    updated_at = NOW()
                WHERE id = %s
                """,
                (first_player["id"], game_id),
            )
            cur.execute(
                "UPDATE players SET actions_left = 3 WHERE game_id = %s",
                (game_id,),
            )
            return first_player

```

```

def action_for_player(game_id: int, user_id: int, action: str) -> Dict[str, Any]:
    costs = {
        "build": 200,
        "research": 120,
        "trade": 0,
        "diplomacy": 100,
    }

    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT * FROM games WHERE id = %s FOR UPDATE", (game_id,))
            game = cur.fetchone()
            if not game or game["status"] != "running":
                raise ValueError("not_running")

            cur.execute("SELECT * FROM players WHERE id = %s AND game_id = %s FOR UPDATE",
(game["current_player_id"], game_id))
            current = cur.fetchone()
            if not current or current["user_id"] != user_id:
                current_name = "Player"
                if current:
                    current_name = current.get("first_name") or current.get("username") or
"Player"

                raise ValueError(f"not_your_turn:{current_name}")

            if current["actions_left"] <= 0:
                raise ValueError("not_enough_actions")
            if action not in costs:
                raise ValueError("unknown_action")

            cost = costs[action]
            if current["money"] < cost:
                raise ValueError(f"not_enough_money:{current['money']}")

            money = current["money"]
            industry = current["industry"]
            science = current["science"]
            stability = current["stability"]
            population = current["population"]

            if action == "build":
                money -= 200
                industry += 2
                population += 1
            elif action == "research":
                money -= 120
                science += 3
            elif action == "trade":
                money += 220

```

```

        stability = max(0, stability - 1)
    elif action == "diplomacy":
        money -= 100
        stability = min(100, stability + 5)

    actions_left = current["actions_left"] - 1
    score = calculate_score_values(money, industry, science, stability, population)

    cur.execute(
        """
        UPDATE players
        SET money = %s,
            industry = %s,
            science = %s,
            stability = %s,
            population = %s,
            actions_left = %s,
            score = %s
        WHERE id = %s
        """,
        (money, industry, science, stability, population, actions_left, score,
current["id"]),
    )

    updated = dict(current)
    updated.update(
        {
            "money": money,
            "industry": industry,
            "science": science,
            "stability": stability,
            "population": population,
            "actions_left": actions_left,
            "score": score,
        }
    )
    return updated


def calculate_score_values(money: int, industry: int, science: int, stability: int, population:
int) -> int:
    return max(0, money) + industry * 50 + science * 100 + stability * 10 + population * 5


def end_turn(game_id: int, user_id: int) -> Dict[str, Any]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT * FROM games WHERE id = %s FOR UPDATE", (game_id,))
            game = cur.fetchone()
            if not game or game["status"] != "running":

```

```

        raise ValueError("not_running")

    cur.execute("SELECT * FROM players WHERE id = %s AND game_id = %s FOR UPDATE",
(game["current_player_id"], game_id))
    current = cur.fetchone()
    if not current or current["user_id"] != user_id:
        current_name = current.get("first_name") if current else "Player"
        raise ValueError(f"not_your_turn:{current_name or 'Player'}")

    # End-of-turn income.
    income = 100 + current["industry"] * 10 + current["population"] * 2
    food_effect = max(-3, min(3, (current["industry"] // 5) - 2))
    money = current["money"] + income
    population = max(1, current["population"] + 1 + food_effect)
    stability = max(0, min(100, current["stability"] + (1 if money >= 1000 else -1)))
    score = calculate_score_values(money, current["industry"], current["science"],
stability, population)

    cur.execute(
        """
        UPDATE players
        SET money = %s,
            population = %s,
            stability = %s,
            actions_left = 3,
            score = %s
        WHERE id = %s
        """,
        (money, population, stability, score, current["id"]),
    )

    cur.execute("SELECT * FROM players WHERE game_id = %s ORDER BY id", (game_id,))
    players = list(cur.fetchall())
    current_index = next(i for i, p in enumerate(players) if p["id"] == current["id"])
    next_index = (current_index + 1) % len(players)
    next_player = players[next_index]

    total_turn = game["total_turn"] + 1
    if total_turn >= 30:
        cur.execute(
            """
            UPDATE players
            SET score = money + (industry * 50) + (science * 100)
                + (stability * 10) + (population * 5)
            WHERE game_id = %s
            """,
            (game_id,),
        )
        cur.execute(
            "UPDATE games SET status = 'finished', total_turn = %s, current_player_id =

```

```

NULL, updated_at = NOW() WHERE id = %s",
        (total_turn, game_id),
    )
    cur.execute(
        "SELECT * FROM players WHERE game_id = %s ORDER BY score DESC, id ASC",
        (game_id,),
    )
    return {
        "finished": True,
        "turn": total_turn,
        "results": list(cur.fetchall()),
        "income": income,
    }

    cur.execute(
        """
        UPDATE games
        SET total_turn = %s,
            current_player_id = %s,
            updated_at = NOW()
        WHERE id = %s
        """,
        (total_turn, next_player["id"], game_id),
    )
    return {
        "finished": False,
        "turn": total_turn,
        "next_player": next_player,
        "income": income,
    }

# =====
# FORMAT GAME
# =====

def format_players(players: List[Dict[str, Any]], language: str) -> str:
    lines = []
    for i, p in enumerate(players, start=1):
        name = p.get("first_name") or (f"@{p['username']}" if p.get("username") else f"Player {i}")
        country = country_name(p.get("country"), language)
        lines.append(f"{i}. {name} – {country}")
    return "\n".join(lines) if lines else "-"

def format_game(game_id: int, language: str) -> str:
    with get_conn() as conn:
        with conn.cursor() as cur:

```

```

cur.execute("SELECT * FROM games WHERE id = %s", (game_id,))
game = cur.fetchone()
if not game:
    return "Game not found."

cur.execute("SELECT * FROM players WHERE game_id = %s ORDER BY id", (game_id,))
players = list(cur.fetchall())

text = []
status_key = game["status"]
status_label = TEXTS[language].get(status_key, status_key)
text.append(f"🎮 <b>{TEXTS[language]['lobby']} if status_key == 'waiting' else
TEXTS[language]['running']}</b>")
text.append(f"<b>Code:</b> <code>{game['code']}</code>")
text.append(f"<b>Status:</b> {status_label}")
text.append(f"<b>{TEXTS[language]['players_title']}</b> {len(players)} / 12")
if game.get("total_turn"):
    text.append(f"<b>{TEXTS[language]['turn']}</b> {game['total_turn']} / 30")

if status_key == "running" and game.get("current_player_id"):
    current = next((p for p in players if p["id"] == game["current_player_id"]), None)
    if current:
        current_name = current.get("first_name") or current.get("username") or "Player"
        text.append(f"<b>Current:</b> {current_name}")

text.append("")
text.append(format_players(players, language))
return "\n".join(text)

def format_player_stats(player: Dict[str, Any], language: str) -> str:
    return (
        f"🌍 <b>{country_name(player.get('country'), language)}</b>\n"
        f"💰 {TEXTS[language]['money']}: <b>{player['money']}</b>\n"
        f"🏢 {TEXTS[language]['industry']}: <b>{player['industry']}</b>\n"
        f"🔬 {TEXTS[language]['science']}: <b>{player['science']}</b>\n"
        f"🛡️ {TEXTS[language]['stability']}: <b>{player['stability']}</b>\n"
        f"👥 {TEXTS[language]['population']}: <b>{player['population']}</b>\n"
        f"🎯 {TEXTS[language]['actions']}: <b>{player['actions_left']}</b>\n"
        f"🏆 Score: <b>{player['score']}</b>"
    )

def format_results(results: List[Dict[str, Any]], language: str) -> str:
    lines = []
    for i, p in enumerate(results, start=1):
        name = p.get("first_name") or p.get("username") or f"Player {i}"
        cname = country_name(p.get("country"), language)
        lines.append(f"{i}. {name} – {cname} – {p['score']}")
    return "\n".join(lines)

```

```

# =====
# TELEGRAM API
# =====

def telegram_call(method: str, payload: Dict[str, Any]) -> Dict[str, Any]:
    if not TELEGRAM_API:
        raise RuntimeError("BOT_TOKEN is missing")

    data = urllib.parse.urlencode({
        k: json.dumps(v, ensure_ascii=False) if isinstance(v, (dict, list)) else str(v)
        for k, v in payload.items()
    }).encode("utf-8")

    req = urllib.request.Request(
        f"{TELEGRAM_API}/{method}",
        data=data,
        headers={"Content-Type": "application/x-www-form-urlencoded; charset=UTF-8"},
        method="POST",
    )

    try:
        with urllib.request.urlopen(req, timeout=20) as response:
            result = json.loads(response.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        body = e.read().decode("utf-8", errors="replace")
        logger.error("Telegram HTTP %s: %s", e.code, body)
        raise

    if not result.get("ok"):
        logger.error("Telegram API error on %s: %s", method, result)
    return result

def send_message(chat_id: int, text: str, reply_markup: Optional[Dict[str, Any]] = None,
parse_mode: str = "HTML"):
    payload: Dict[str, Any] = {"chat_id": chat_id, "text": text}
    if parse_mode:
        payload["parse_mode"] = parse_mode
    if reply_markup is not None:
        payload["reply_markup"] = reply_markup
    return telegram_call("sendMessage", payload)

def edit_message(chat_id: int, message_id: int, text: str, reply_markup: Optional[Dict[str,
Any]] = None):
    payload: Dict[str, Any] = {
        "chat_id": chat_id,

```

```

        "message_id": message_id,
        "text": text,
        "parse_mode": "HTML",
    }
    if reply_markup is not None:
        payload["reply_markup"] = reply_markup
    return telegram_call("editMessageText", payload)

def answer_callback(callback_id: str, text: Optional[str] = None, show_alert: bool = False):
    payload: Dict[str, Any] = {"callback_query_id": callback_id, "show_alert":
str(show_alert).lower()}
    if text:
        payload["text"] = text
    return telegram_call("answerCallbackQuery", payload)

def set_webhook():
    if not WEBHOOK_URL:
        raise RuntimeError("WEBHOOK_URL is missing")
    endpoint = f"{WEBHOOK_URL}/telegram"
    payload = {"url": endpoint}
    if WEBHOOK_SECRET:
        payload["secret_token"] = WEBHOOK_SECRET
    return telegram_call("setWebhook", payload)

# =====
# KEYBOARDS
# =====

def main_keyboard(language: str):
    if language == "fa":
        return {
            "keyboard": [
                [{"text": TEXTS[language]["create_button"]}, {"text": TEXTS[language]
["join_button"]}],
                [{"text": TEXTS[language]["my_game"]}, {"text": TEXTS[language]["game_menu"]}],
                [{"text": TEXTS[language]["rules_button"]}, {"text": TEXTS[language]
["help_button"]}],
                [{"text": TEXTS[language]["language_button"]}],
            ],
            "resize_keyboard": True,
        }
    return {
        "keyboard": [
            [{"text": TEXTS[language]["create_button"]}, {"text": TEXTS[language]
["join_button"]}],
            [{"text": TEXTS[language]["my_game"]}, {"text": TEXTS[language]["game_menu"]}],

```

```

        [{"text": TEXTS[language]["rules_button"]}, {"text": TEXTS[language]
["help_button"]}],
        [{"text": TEXTS[language]["language_button"]}],
    ],
    "resize_keyboard": True,
}

```

```

def language_keyboard():
    return {
        "inline_keyboard": [
            [{"text": "فارسی 🇮🇷", "callback_data": "lang:fa"}, {"text": "English 🇬🇧",
"callback_data": "lang:en"}]
        ]
    }

```

```

def lobby_keyboard(game: Dict[str, Any], user_id: int, language: str):
    buttons = [
        [{"text": TEXTS[language]["choose_country"], "callback_data": f"country:{game['id']}"},
        [{"text": TEXTS[language]["my_game"], "callback_data": f"refresh:{game['id']}"},
    ]
    if game.get("owner_id") == user_id:
        buttons.insert(0, [{"text": f"🚀 {TEXTS[language]['start_game']}", "callback_data":
f"start:{game['id']}"})
    return {"inline_keyboard": buttons}

```

```

def country_keyboard(game_id: int, language: str):
    rows = []
    items = list(COUNTRIES.items())
    for i in range(0, len(items), 2):
        row = []
        for code, data in items[i : i + 2]:
            row.append({"text": data[language], "callback_data": f"pickcountry:{game_id}:
{code}"})
        rows.append(row)
    rows.append([{"text": TEXTS[language]["back"], "callback_data": f"refresh:{game_id}"})
    return {"inline_keyboard": rows}

```

```

def running_keyboard(game_id: int, language: str):
    return {
        "inline_keyboard": [
            [
                {"text": TEXTS[language]["build"], "callback_data": f"action:{game_id}:build"},
                {"text": TEXTS[language]["research"], "callback_data": f"action:
{game_id}:research"},
            ],
            [

```

```

        {"text": TEXTS[language]["trade"], "callback_data": f"action:{game_id}:trade"},
        {"text": TEXTS[language]["diplomacy"], "callback_data": f"action:
{game_id}:diplomacy"},
    ],
    [{"text": TEXTS[language]["end_turn"], "callback_data": f"endturn:{game_id}"},
    [{"text": TEXTS[language]["my_game"], "callback_data": f"refresh:{game_id}"},
    ]
}

```

```

# =====
# GAME MESSAGE RENDERING
# =====

```

```

def send_game_status(chat_id: int, game_id: int, user_id: int):
    language = get_language(user_id)
    game_text = format_game(game_id, language)
    game = get_game_by_id(game_id)
    if not game:
        send_message(chat_id, t(user_id, "game_not_found"), main_keyboard(language))
        return

```

```

    player = get_player(game_id, user_id)
    if game["status"] == "waiting":
        markup = lobby_keyboard(game, user_id, language)
    elif game["status"] == "running":
        markup = running_keyboard(game_id, language)
    else:
        markup = main_keyboard(language)

    if player and game["status"] == "running":
        game_text += "\n\n" + format_player_stats(player, language)
    send_message(chat_id, game_text, markup)

```

```

def get_game_by_id(game_id: int) -> Optional[Dict[str, Any]]:
    with get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("SELECT * FROM games WHERE id = %s", (game_id,))
            return cur.fetchone()

```

```

# =====
# HANDLERS
# =====

```

```

def handle_start(message: Dict[str, Any]):
    user = message["from"]

```

```

user_id = int(user["id"])
chat_id = int(message["chat"]["id"])
language = get_language(user_id)

set_pending_action(user_id, None)
text = t(user_id, "welcome", name=display_name(user))
if message["chat"].get("type") == "private":
    text += "\n\n" + t(user_id, "private_tip")
send_message(chat_id, text, main_keyboard(language))

def handle_text(message: Dict[str, Any]):
    user = message["from"]
    user_id = int(user["id"])
    chat = message["chat"]
    chat_id = int(chat["id"])
    text = (message.get("text") or "").strip()
    language = get_language(user_id)

    if text.startswith("/start"):
        handle_start(message)
        return

    if text.startswith("/help"):
        send_message(chat_id, t(user_id, "help"), main_keyboard(language))
        return

    if text.startswith("/rules"):
        send_message(chat_id, t(user_id, "rules"), main_keyboard(language))
        return

    if text.startswith("/game") or text == TEXTS[language]["my_game"]:
        game = get_user_game(user_id)
        if not game:
            send_message(chat_id, t(user_id, "no_game"), main_keyboard(language))
        else:
            send_game_status(chat_id, game["id"], user_id)
        return

    if text == TEXTS[language]["language_button"] or text == "/language":
        send_message(chat_id, t(user_id, "choose_language"), language_keyboard())
        return

    if text == TEXTS[language]["rules_button"]:
        send_message(chat_id, t(user_id, "rules"), main_keyboard(language))
        return

    if text == TEXTS[language]["help_button"]:
        send_message(chat_id, t(user_id, "help"), main_keyboard(language))
        return

```

```

if text == TEXTS[language]["create_button"] or text == "/newgame":
    if chat.get("type") == "private":
        send_message(chat_id, t(user_id, "need_group"), main_keyboard(language))
        return
    try:
        game = create_game(user, chat_id)
        send_message(
            chat_id,
            t(user_id, "game_created", code=game["code"], name=display_name(user)),
            lobby_keyboard(game, user_id, language),
        )
    except ValueError as exc:
        if str(exc) == "already_in_game":
            game = get_user_game(user_id)
            send_message(chat_id, t(user_id, "game_already", code=game["code"]),
main_keyboard(language))
        else:
            logger.exception("Could not create game")
            send_message(chat_id, "Database/game error. Please try again.",
main_keyboard(language))
        return

if text == TEXTS[language]["join_button"] or text == "/join":
    set_pending_action(user_id, "join_game")
    send_message(chat_id, t(user_id, "join_enter_code"), main_keyboard(language))
    return

pending = get_pending_action(user_id)
if pending == "join_game":
    code = text.upper()
    if len(code) != 6 or not all(c in string.ascii_uppercase + string.digits for c in code):
        send_message(chat_id, t(user_id, "invalid_code"), main_keyboard(language))
        return

    game = get_game_by_code(code)
    if not game or game["status"] != "waiting":
        set_pending_action(user_id, None)
        send_message(chat_id, t(user_id, "game_not_found"), main_keyboard(language))
        return

    try:
        join_game(game["id"], user, chat_id)
        set_pending_action(user_id, None)
        refreshed = get_game_by_id(game["id"])
        send_message(
            chat_id,
            t(user_id, "joined", game=format_game(game["id"], language)),
            lobby_keyboard(refreshed, user_id, language),
        )

```

```

except ValueError as exc:
    err = str(exc)
    set_pending_action(user_id, None)
    if err == "already_in_game":
        send_message(chat_id, t(user_id, "already_in_game"), main_keyboard(language))
    elif err == "game_full":
        send_message(chat_id, "Game is full.", main_keyboard(language))
    else:
        send_message(chat_id, t(user_id, "game_not_found"), main_keyboard(language))
return

if text == TEXTS[language]["game_menu"]:
    game = get_user_game(user_id)
    if not game:
        send_message(chat_id, t(user_id, "no_game"), main_keyboard(language))
    else:
        send_game_status(chat_id, game["id"], user_id)
    return

# Command shortcuts.
if text.startswith("/status"):
    game = get_user_game(user_id)
    if game:
        send_game_status(chat_id, game["id"], user_id)
    else:
        send_message(chat_id, t(user_id, "no_game"), main_keyboard(language))
    return

send_message(chat_id, t(user_id, "unknown"), main_keyboard(language))

def handle_callback(callback: Dict[str, Any]):
    callback_id = callback["id"]
    user = callback["from"]
    user_id = int(user["id"])
    data = callback.get("data", "")
    message = callback.get("message") or {}
    chat = message.get("chat") or {}
    chat_id = int(chat.get("id", user_id))
    message_id = int(message.get("message_id", 0)) if message.get("message_id") else None
    language = get_language(user_id)

    try:
        if data.startswith("lang:"):
            selected = data.split(":", 1)[1]
            set_language(user_id, selected)
            language = selected
            answer_callback(callback_id, t(user_id, "language_saved"))
            send_message(chat_id, t(user_id, "welcome", name=display_name(user)),
main_keyboard(language))

```

```

        return

    if data.startswith("country:"):
        game_id = int(data.split(":")[1])
        answer_callback(callback_id)
        game = get_game_by_id(game_id)
        if not game:
            send_message(chat_id, t(user_id, "game_not_found"), main_keyboard(language))
            return
        if game["status"] != "waiting":
            send_message(chat_id, t(user_id, "not_running"), main_keyboard(language))
            return
        send_message(chat_id, t(user_id, "country_open"), country_keyboard(game_id,
language))
        return

    if data.startswith("pickcountry:"):
        _, game_id_str, code = data.split(":", 2)
        game_id = int(game_id_str)
        try:
            ok = choose_country(game_id, user_id, code)
        except ValueError as exc:
            if str(exc) == "country_taken":
                answer_callback(callback_id, t(user_id, "country_taken"), True)
                return
            raise
        if not ok:
            answer_callback(callback_id, t(user_id, "game_not_found"), True)
            return
        answer_callback(callback_id, t(user_id, "country_saved", country=country_name(code,
language)))
        if message_id:
            edit_message(chat_id, message_id, t(user_id, "country_open"),
country_keyboard(game_id, language))
            send_game_status(chat_id, game_id, user_id)
            return

    if data.startswith("start:"):
        game_id = int(data.split(":")[1])
        try:
            first_player = start_game(game_id, user_id)
        except ValueError as exc:
            key = str(exc)
            if key == "not_owner":
                answer_callback(callback_id, t(user_id, "not_owner"), True)
                return
            if key == "need_players":
                answer_callback(callback_id, t(user_id, "need_players"), True)
                return
            if key == "need_country":

```

```

        answer_callback(callback_id, t(user_id, "need_country"), True)
        return
    if key == "not_running":
        answer_callback(callback_id, t(user_id, "not_running"), True)
        return
    raise
    answer_callback(callback_id, t(user_id, "game_started",
player=first_player.get("first_name") or first_player.get("username") or "Player"))
    refreshed = get_game_by_id(game_id)
    public_text = t(user_id, "game_started", player=first_player.get("first_name") or
first_player.get("username") or "Player")
    public_text += "\n\n" + format_game(game_id, language)
    send_message(chat_id, public_text, running_keyboard(game_id, language))
    return

if data.startswith("action:"):
    _, game_id_str, action = data.split(":", 2)
    game_id = int(game_id_str)
    try:
        player = action_for_player(game_id, user_id, action)
    except ValueError as exc:
        err = str(exc)
        if err.startswith("not_your_turn:"):
            current_name = err.split(":", 1)[1]
            answer_callback(callback_id, t(user_id, "not_your_turn",
player=current_name), True)
            return
        if err.startswith("not_enough_money:"):
            money = err.split(":", 1)[1]
            answer_callback(callback_id, t(user_id, "not_enough_money", money=money),
True)

            return
        if err == "not_enough_actions":
            answer_callback(callback_id, t(user_id, "not_enough_actions"), True)
            return
        if err == "not_running":
            answer_callback(callback_id, t(user_id, "not_running"), True)
            return
        raise

    answer_callback(callback_id, t(user_id, "action_done",
summary=format_player_stats(player, language)))
    send_game_status(chat_id, game_id, user_id)
    return

if data.startswith("endturn:"):
    game_id = int(data.split(":")[1])
    try:
        result = end_turn(game_id, user_id)
    except ValueError as exc:

```

```

        err = str(exc)
        if err.startswith("not_your_turn:"):
            current_name = err.split(":", 1)[1]
            answer_callback(callback_id, t(user_id, "not_your_turn",
player=current_name), True)
            return
        if err == "not_running":
            answer_callback(callback_id, t(user_id, "not_running"), True)
            return
        raise

    if result["finished"]:
        answer_callback(callback_id)
        game = get_game_by_id(game_id)
        players = get_players(game_id)
        for p in players:
            lang = get_language(p["user_id"])
            text = t(p["user_id"], "game_finished",
results=format_results(result["results"], lang))
            send_message(p["chat_id"] or chat_id, text, main_keyboard(lang))
        return

        answer_callback(callback_id)
        next_name = result["next_player"].get("first_name") or
result["next_player"].get("username") or "Player"
        public_text = t(user_id, "turn_ended", turn=result["turn"], player=next_name)
        public_text += "\n\n" + format_game(game_id, language)
        send_message(chat_id, public_text, running_keyboard(game_id, language))
        return

    if data.startswith("refresh:"):
        game_id = int(data.split(":")[1])
        answer_callback(callback_id)
        send_game_status(chat_id, game_id, user_id)
        return

        answer_callback(callback_id)
except Exception:
    logger.exception("Callback handler failed")
    try:
        answer_callback(callback_id, "Something went wrong.", True)
    except Exception:
        pass

# =====
# FLASK ROUTES
# =====

```

```

@app.get("/")
def home():
    return jsonify({
        "ok": True,
        "service": "strategy-game",
        "message": "Bot is running",
    })

@app.get("/health")
def health():
    db_ok = False
    error = None
    try:
        with get_conn() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1 AS ok")
                db_ok = bool(cur.fetchone()["ok"] == 1)
    except Exception as exc:
        error = str(exc)

    payload = {
        "status": "ok" if db_ok else "degraded",
        "database": db_ok,
        "bot_token_configured": bool(BOT_TOKEN),
        "webhook_url_configured": bool(WEBHOOK_URL),
    }
    if error:
        payload["database_error"] = error
    return jsonify(payload), (200 if db_ok else 503)

@app.route("/set-webhook", methods=["GET", "POST"])
def webhook_setup():
    if ADMIN_KEY:
        supplied = request.headers.get("X-Admin-Key", "")
        if supplied != ADMIN_KEY:
            return jsonify({"ok": False, "error": "Forbidden"}), 403

    try:
        result = set_webhook()
        return jsonify(result)
    except Exception as exc:
        logger.exception("Webhook setup failed")
        return jsonify({"ok": False, "error": str(exc)}), 500

@app.post("/telegram")
def telegram_webhook():
    if WEBHOOK_SECRET:

```

```

incoming_secret= request.headers.get("X-Telegram-Bot-API-Secret-Token", "")
if incoming_secret != WEBHOOK_SECRET:
    return jsonify({"ok": False, "error": "Unauthorized"}), 403

update = request.get_json(silent=True) or {}

try:
    if update.get("callback_query"):
        handle_callback(update["callback_query"])
    elif update.get("message"):
        handle_text(update["message"])
    elif update.get("edited_message"):
        handle_text(update["edited_message"])
except Exception:
    logger.exception("Update handling failed")

return jsonify({"ok": True})

# =====
# STARTUP
# =====

def startup():
    if not DATABASE_URL:
        logger.error("DATABASE_URL is missing")
    else:
        try:
            init_db()
        except Exception:
            logger.exception("Database initialization failed")

    if not BOT_TOKEN:
        logger.error("BOT_TOKEN is missing")
    else:
        logger.info("Telegram bot token is configured")

startup()

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=PORT)

```